



WORKSHOP 12

Cost Model

CSX3004 Programming Languages
Kwankamol Nongpong

TASK 1: COMPUTE THE COST

```
sum_list(alist):  
    total = 0  
    for x in alist:  
        total = total + x  
    return total
```

The cost is defined as "number of additions".

Task: Compute cost for the list of size n

TASK 2: COMPUTE THE COST

```
factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

The cost is defined as "number of multiplications + number of recursive calls".

Task: Express cost recurrence.

TASK 3: COST COMPARISON

% version 1

rev([], []).

rev([H|T], R) :- rev(T, RT), append(RT, [H], R).

% version 2

rev(L, R) :- rev_helper(L, [], R).

rev_helper([], Sofar, Sofar).

rev_helper([H|T], Sofar, R) :- rev_helper(T, [H|Sofar], R).

TASK 3 (CONTINUED)

The cost is defined as the number of append calls.

1. Derive cost equations of each version.
2. Compute cost for Version 1 and Version 2.
3. Which version is more efficient? Why?

TASK 4

Consider the code given in Task 2 (Factorial) and Task 3 (List Reversal).

Suppose the cost model counts function calls instead of operations.

1. How does the cost of factorial change?
2. How does the cost of list reversal (Version 1 vs. Version 2) change?

TASK 5: ROW-MAJOR VS. COLUMN-MAJOR ORDER

Consider the C code (task5.c) given in the Workshop 12 assignment and answer the following questions.

1. Which loop order will be faster in C? Why?
2. Compile and execute the program. Record both timings.
3. What would happen if the same code was written in Fortran or MATLAB?
4. Modify the program to use **dynamic allocation** (malloc) and see if results change.
5. Write equivalent code in Python and test using NumPy.